Intitulé du cours:
Technique des applications web

Chapitre 4 :
props & State en React

Enseignante: Abid Arij

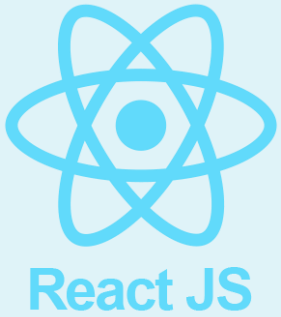
Année de formation 2025-2026

React JS

Plan

- ❑Props en react
- ❑ State en react

props



props

- Abréviation de « **properties** » pour contenir les informations relatives à un composant.
- Technique qui consiste à partager du code **entre des composants React**
- La plupart des composants peuvent être personnalisés avec différents props lors de leur création.

- A la base, props est un **objet.** Il stocke les valeurs des attributs (Attributes) d'une étiquette (Tag) :
- arguments transmis aux composants React via des attributs Html
- arguments transmis par le parent et accessibles via : this.props
- **Props** sont des **données envoyées d'un composant parent vers un composant enfant**.

props

- Exemple :

Composant Enfant

personne.js

```
import React from 'react';
class Personne extends React.Component {
    render() {
        return (<div>
                    <h2>Prénom : {this.props.prenom} </h2>
                    <h2>Nom : {this.props.nom} </h2>
                </div>) }
} export default Personne;
```

Composant Parent

App.js

```
import Personne from './personne';
class App extends React.Component {
  render() {
    const myElement=<Personne prenom = " arij " nom = " abid "/>;
    return (
      <div> {myElement} </div>
    ); }
}
```

- **Remarques importantes :**

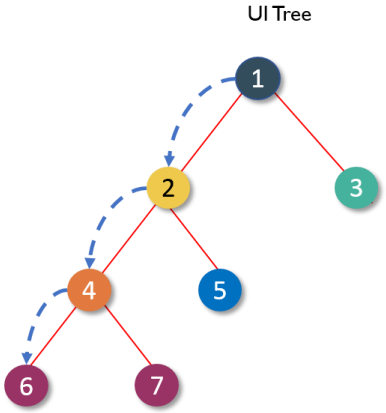
- **props :** propriétés immuables : on ne peut pas affecter des valeurs ou des objets à '**props**' (chose qu'on peut faire par contre au niveau de '**state**').
  - On peut affecter des valeurs par défaut aux ***props*** via : **defaultProps**

personne.js

```
import React from 'react';
class Personne extends React.Component {
    render() {
        return (<div>
                    <h2>Prénom : {this.props.prenom} </h2>
                    <h2>Nom : {this.props.nom} </h2>
                </div>) }
}
Personne.defaultProps = { prenom : "Foulen",
                          nom : "Foulenii" }
export default Personne;
```

```
<Personne prenom = "Anis" nom = "ASSAS"/>
<Personne prenom = "Ahmed"/>
<Personne nom = "Ben Ahmed"/>
<Personne />
```

props



props (properties) sont des **données envoyées d'un composant parent vers un composant enfant**.

- Elles sont **en lecture seule (immutable)**.
- Le composant enfant **ne peut pas les modifier**.
- Elles servent à **passer des informations entre composants**.

```
function App() {
  return (
    <div>
      <Product name="Laptop" price={1200} />
      <Product name="Phone" price={800} />
    </div>
  );
}
```

```
function Product({ name, price }) {
  return (
    <div>
      <h3>{name}</h3>
      <p>{price} DT</p>
    </div>
  );
}
```

props

Caractéristiques :

- **Lecture seule (read-only)** → on ne peut pas les modifier dans le composant enfant.
- Servent à **passer des informations** entre composants.
- Rendent les composants **réutilisables**.

Exemple :

```
// Composant enfant — reçoit des props
function Carte({ nom, age }) {
  return (
    <div>
      <h2>Bonjour, je suis {nom}</h2>
      <p>J'ai {age} ans</p>
    </div>
  );
}
```

```
// Composant parent — envoie des props
function App() {
  return (
    <div>
      <Carte nom="Alice" age={25} />
      <Carte nom="Bob" age={30} />
    </div>
  );
}
```

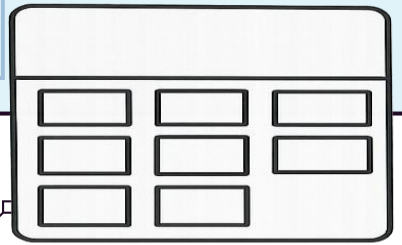
💡 Les props c'est comme remplir un formulaire. Tu passes les infos, le composant les affiche. Tu ne peux pas changer le formulaire de l'intérieur.

React JS

State

<Component />

```
const [count, setCount] = useState(0);
```



<useState>

state

State en React

State = état interne du composant
Le **state** est une **variable interne d'un composant React** qui peut changer au fil du temps.
Quand le **state change**, React **met à jour automatiquement l'interface (le rendu)**.
Une variable normale JavaScript ne fait jamais ça. C'est la seule différence, mais elle est énorme !

- **Syntaxe :**

```
const [variable, setVariable] = useState(valeurInitiale)
```

Élément	Signification
variable	la variable d'état (en lecture seule).
setVariable	la fonction pour modifier l'état et déclencher un re-rendu.
useState()	hook React

State en React

- On importe le hook useState depuis React, qui permet de gérer un **état local** dans le composant.

```
import { useState } from "react";

function Counter() {

  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Compteur : {count} </h2>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
export default Counter;
```

- **count** → la variable qui stocke la valeur actuelle du compteur (commence à **0**)
- **setCount** → la fonction pour **modifier** cette valeur
- **useState(0)** → initialise le compteur à **0**

quand on clique, on appelle setCount(count + 1) ce qui **ajoute 1** au compteur React re-affiche automatiquement le composant à chaque changement de count

Rend le composant utilisable dans d'autres fichiers.

Quand on utilise useState ?

On l'utilise chaque fois qu'on veut **une donnée qui peut changer et qui doit mettre à jour l'interface automatiquement**.

Exemples typiques :

- Compteur simple
- Texte saisi par l'utilisateur
- Afficher ou cacher un élément (toggle)
- Liste dynamique

Utilité	Exemple
Compteur qui change	const [count, setCount] = useState(0)
Formulaire utilisateur	const [nom, setNom] = useState("")
Affichage conditionnel	const [show, setShow] = useState(true)
Liste dynamique	const [taches, setTaches] = useState([])

Les types de State

- **Nombre**

```
const [age, setAge] = useState(0);
setAge(25);
```

- **Texte (string)**

```
const [prenom, setPrenom] = useState("");
setPrenom("Alice");
```

- **Booléen (true/false)**

```
const [estConnecte, setEstConnecte] = useState(false);
setEstConnecte(true);
```

Les types de State

➢ **Tableau (liste)**

const [fruits, setFruits] = useState(["pomme", "banane"]);

❖ **Ajouter un élément**

setFruits([...fruits, "orange"]); // les éléments sont extraits
// → ["pomme", "banane", "orange"]

... C'est l'opérateur **spread** (décomposition). Il **"étale"** les éléments d'un tableau.

Pourquoi on utilise ... ?
En React ❗
👉 On **ne modifie jamais directement le state**

`...fruits` = copie les éléments
`[...]` = crée un nouveau tableau
`setFruits()` = met à jour le state

❌ Mauvais (interdit) :
fruits.push("orange");
setFruits(fruits);

✅ Bon :
setFruits([...fruits, "orange"]);
👉 Parce que React a besoin d'un **nouvel objet** pour détecter le changement.

Les types de State

➢ **Objet**

```
const [utilisateur, setUtilisateur] = useState({ nom: "Alice", age: 25 });

// Modifier une propriété
setUtilisateur({ ...utilisateur, age: 26 });
```

Le . . . sert à **copier les éléments d'un tableau (ou objet)** sans modifier l'original.

❑ **Problème sans ...**

Si tu fais ça :

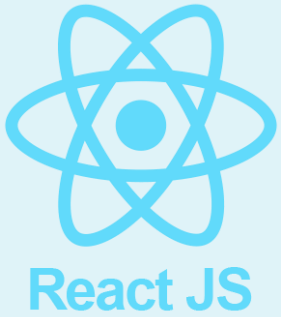
```
fruits.push("orange");
setFruits(fruits);
```

👉 Tu modifies directement le tableau original ❌

⚠️ **En React :**

le state doit être **immutable (non modifié directement)**
sinon React **ne détecte pas bien les changements**

Hook



Hook

- Un Hook est une fonction speciale de React qui commence toujours par le mot **use**.
- Les Hooks te permettent d'ajouter des fonctionnalites puissantes dans tes composants sans ecrire de classes.

Règles des hooks

- Appeler uniquement dans des composants fonctionnels ou des custom hooks
- Ne pas appeler dans des conditions, boucles ou fonctions imbriquées
- Toujours au niveau supérieur du composant (top-level)

Cliquez sur un hook pour avoir des exemples détaillés

Les hooks React

État (State)

useState
Valeur locale dans le composant

useReducer
État complexe via actions

Effets (Side effects)

useEffect
Fetch, timers, abonnements

useLayoutEffect
Avant le rendu du DOM

Contexte & Refs

useContext
Données globales partagées

useRef
Accès DOM ou valeur persistante

Performance

useMemo
Mémoïse une valeur calculée

useCallback
Mémoïse une fonction

Avancés

useImperativeHandle
Exposer des méthodes via ref

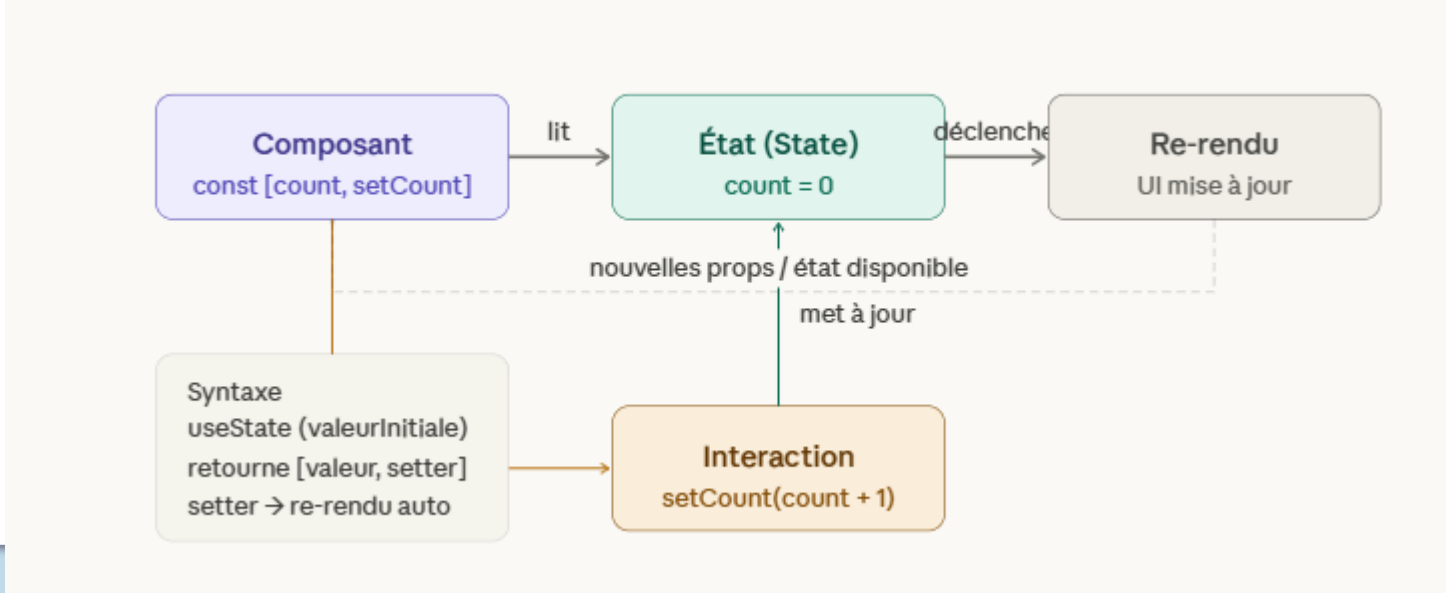
useId
ID unique côté client et serveur

Hooks personnalisés

Custom hooks
Réutiliser de la logique entre composants
ex: useFetch, useDebounce...

useState

- Le principe clé : chaque appel à **setCount** provoque un re-rendu, et React récupère la nouvelle valeur au prochain rendu. L'état est **persistant** entre les rendus (contrairement à une simple variable) mais **isolé** par composant.
- Voici maintenant une démo interactive avec plusieurs cas d'usage concrets :



Merci pour

votre Attetion